inject it into your classes and endpoint handlers. You could rewrite the endpoint handler in listing 10.3 as the following, and the `IConfiguration` object would be injected into the handler using DI:

```
app.MapGet("/", (IConfiguration config) => config.AsEnumerable());
```

NOTE The `ConfigurationManager` implements `IConfigurationRoot`, which also implements `IConfiguration`. The `ConfigurationManager` is registered in the DI container as an `IConfiguration`, not an `IConfigurationRoot`.

You've seen how to add values to the `ConfigurationManager` by using providers such as the JSON file provider, and listing 10.3 showed an example of iterating over every configuration value, but normally you want to retrieve a specific configuration value.

`IConfiguration` stores configuration as a set of key-value `string` pairs. You can access any value by its key, using standard dictionary syntax. You could use

```
var zoomLevel = builder.Configuration["MapSettings:DefaultZoomLevel"];
```

to retrieve the configured zoom level for your application (using the settings shown in listing 10.2). Note that I used a colon (`:`) to designate a separate section. Similarly, to retrieve the `latitude` key, you could use

```
var lat = builder.Configuration["MapSettings:DefaultLocation:Latitude"];
```

NOTE If the requested configuration key doesn't exist, you get a `null` value.

You can also grab a whole section of the configuration by using the `GetSection (section)` method, which returns an `IConfigurationSection,` which also implements